



TeleStroke: real-time stroke detection with federated learning and YOLOv8 on edge devices

Abdussalam Elhanashi¹ · Pierpaolo Dini¹ · Sergio Saponara¹ · Qinghe Zheng²

Received: 25 March 2024 / Accepted: 15 June 2024 / Published online: 26 June 2024
© The Author(s) 2024

Abstract

Stroke, a life-threatening medical condition, necessitates immediate intervention for optimal outcomes. Timely diagnosis and treatment play a crucial role in reducing mortality and minimizing long-term disabilities associated with strokes. This study presents a novel approach to meet these critical needs by proposing a real-time stroke detection system based on deep learning (DL) with utilization of federated learning (FL) to enhance accuracy and privacy preservation. The primary objective of this research is to develop an efficient and accurate model capable of discerning between stroke and non-stroke cases in real-time, facilitating healthcare professionals in making well-informed decisions. Traditional stroke detection methods relying on manual interpretation of medical images are time-consuming and prone to human error. DL techniques have shown promise in automating this process, yet challenges persist due to the need for extensive and diverse datasets and privacy concerns. To address these challenges, our methodology involves utilization and assessing YOLOv8 models on comprehensive datasets comprising both stroke and non-stroke based on the facial paralysis of the individuals from the images. This training process empowers the model to grasp intricate patterns and features associated with strokes, thereby enhancing its diagnostic accuracy. In addition, federated learning, a decentralized training approach, is employed to bolster privacy while preserving model performance. This approach enables the model to learn from data distributed across various clients without compromising sensitive patient information. The proposed methodology has been implemented on NVIDIA platforms, utilizing their advanced GPU capabilities to enable real-time processing and analysis. This optimized model has the potential to revolutionize stroke diagnosis and patient care, promising to save lives and elevate the quality of healthcare services in the neurology field.

Keywords Stroke detection · Federated learning · Deep learning · Real time · Healthcare · NVIDIA platforms

1 Introduction

Stroke is a leading cause of death and disability worldwide, making early detection and diagnosis crucial for improving patient outcomes. Rapid and accurate identification of stroke symptoms, along with timely diagnostic testing, are essential for initiating appropriate treatment and minimizing long-term consequences. This introduction will provide an overview of stroke detection and diagnosis, including the signs and symptoms of stroke, diagnostic tools and

techniques, and the importance of early intervention [1–4]. Stroke, also known as a cerebrovascular accident, occurs when the blood supply to the brain is interrupted, leading to damage or death of brain cells. There are two main types of strokes: ischemic stroke, caused by a blockage in a blood vessel supplying the brain, and hemorrhagic stroke, caused by bleeding into the brain. The common signs and symptoms of stroke can be remembered using the acronym FAST: face drooping, arm weakness, speech difficulty, time to call emergency services. Other symptoms include sudden numbness or weakness in the face, arm, or leg, especially on one side of the body; sudden confusion, trouble speaking or understanding speech; sudden trouble seeing in one or both eyes; sudden trouble walking, dizziness, loss of balance or coordination; and sudden severe headache with no known cause [5]. When a patient presents with symptoms suggestive of a stroke, healthcare providers must act quickly to confirm the

✉ Abdussalam Elhanashi
abdussalam.elhanashi@ing.unipi.it

¹ Ingegneria Informazione, University of Pisa, Pisa, Italy

² School of Intelligent Engineering, Shandong Management University, Jinan 250357, Shandong, China

diagnosis and determine the type of stroke in order to initiate appropriate treatment. Several diagnostic tools and techniques are used in the evaluation of stroke patients. These include imaging studies such as computed tomography (CT) scans, magnetic resonance imaging (MRI), and angiography to visualize the brain to assess electrical activity in the brain; and blood tests to evaluate for potential causes of stroke such as high cholesterol, clotting disorders, or infection [6]. Early intervention is critical in the management of stroke as it can help minimize brain damage and improve patient outcomes. The “time is brain” concept emphasizes the importance of rapid assessment and treatment to preserve brain function. For ischemic strokes, timely administration of thrombolytic therapy (such as tissue plasminogen activator) or endovascular clot retrieval can help restore blood flow to the affected area of the brain. In cases of hemorrhagic stroke, prompt neurosurgical intervention would be necessary to control bleeding and reduce pressure on the brain. Despite advancements in stroke diagnosis, current diagnostic tools, such as CT and MRI, often fail to detect minor strokes and differentiate between ischemic and hemorrhagic strokes in the acute phase. There is also a lack of portable, rapid, and cost-effective diagnostic devices for use in pre-hospital settings, where early detection is crucial. Existing biomarkers lack specificity and sensitivity, limiting their clinical utility. The role of telemedicine in stroke diagnosis is underexplored, especially in remote areas with limited access to advanced medical facilities. It is necessary to bridge these gaps and improve diagnostic accuracy and timeliness, ultimately enhancing patient outcomes and reducing the burden on healthcare systems. Therefore, accurate and timely diagnosis is essential for guiding appropriate interventions and improving patient prognosis [7, 8]. In recent years, the rapid advancement of artificial intelligence (AI) and deep learning technologies has revolutionized various industries, including healthcare. These cutting-edge technologies have shown great promise in transforming the way medical diagnostics are conducted and in enhancing the delivery of e-healthcare services. By leveraging AI and deep learning, healthcare professionals can harness the power of data-driven insights to improve diagnostic accuracy, optimize treatment plans, and provide personalized care to patients. This introduction will explore the utilization of AI and deep learning in diagnostics and e-healthcare, highlighting their potential benefits and implications [9]. The integration of AI in healthcare has significantly impacted the way medical professionals approach diagnosis and treatment. AI algorithms have demonstrated the ability to analyze complex medical data, such as imaging scans, genetic information, and patient records, with remarkable speed and accuracy. This has led to the development of advanced diagnostic tools that can assist clinicians in detecting and predicting various medical

conditions, ranging from cancer and cardiovascular diseases to neurological disorders [10].

The main contributions of this research are as the following:

1. The research introduces a novel approach to addressing the urgent need for timely stroke diagnosis based on facial paralysis acquired from the images by presenting a real-time stroke detection system. The proposed approach automates the process of stroke detection, potentially reducing the time required for diagnosis and treatment initiation.
2. This study emphasizes the importance of comprehensive datasets comprising both stroke and non-stroke faces of individuals for effective model training. By utilizing the YOLOv8 models, the study leverages their advanced architectural improvements to enhance real-time processing capabilities. Extensive training on diverse datasets enables the model to discern intricate patterns and features associated with strokes and non-stroke cases, thereby improving its accuracy and responsiveness in real-time applications. The improvements in YOLOv8, such as optimized network structures and enhanced computational efficiency, are crucial for meeting the real-time requirements of stroke detection systems.
3. This research adopts a federated learning technique, allowing the model to learn from data distributed across various clients without compromising patient privacy. By decentralizing the training process, sensitive patient information remains localized, addressing privacy concerns while maintaining model performance.
4. Implementation of the proposed methodology on NVIDIA platforms showcases the practical feasibility of real-time stroke detection. By harnessing advanced GPU capabilities, the system demonstrates remarkable speed and accuracy, revolutionizing stroke diagnosis and treatment with its efficient analysis.

The rest of the paper is organized as follows: Sect. 2 presents the related work; Sect. 3 presents the proposed algorithm and design methodology; Sect. 4 explores the experimental results and discussion. Section 5 describes real-time implementation on edge NVIDIA platforms. Finally, conclusions are drawn in Sect. 6.

2 Related work

Imaging modalities such as computed tomography (CT) and magnetic resonance imaging (MRI) play a pivotal role in the diagnosis of stroke. CT scans are commonly used to rule out hemorrhagic stroke, while MRI is more sensitive

in detecting ischemic stroke, aiding in the differentiation from non-stroke conditions. Deep learning methodologies have demonstrated substantial efficacy in the fields of image and video recognition, revolutionizing the way that computers interpret visual data [11–13]. Deep learning has revolutionized the fields of classification and modulation, offering robust performance gains over traditional methods [14]. Advanced imaging techniques such as perfusion imaging and diffusion-weighted imaging have shown promise in enhancing the accuracy of stroke diagnosis [15].

Utilizing deep learning for medical images has revolutionized diagnostic accuracy and patient care. Deep learning, particularly through convolutional neural networks (CNNs), has shown exceptional promise in various medical imaging applications, including ophthalmology, respiratory imaging, and breast disease. Research has documented instances of medical conditions such as diabetic retinopathy, lung nodules, and breast cancer being diagnosed through the use of medical imaging techniques, including retinal fundus photography, CT scans, and mammograms. Despite these advancements, challenges persist, such as the need for robust quantitative imaging biomarkers, concerns about data quality and bias, and the black-box nature of deep learning models. Efforts are underway to address these challenges and enhance the clinical impact of deep learning in medical imaging [16–18].

Deep learning models have the capacity to forecast patient outcomes, tailor treatment strategies, and streamline administrative operations within telemedicine frameworks. By incorporating deep learning into telemedicine services, healthcare professionals are enabled to deliver care that is both more efficient and economical, effectively overcoming the challenges posed by geographic limitations. This approach ensures that patients receive high-quality care tailored to their specific needs, regardless of where they are located, making healthcare more accessible and personalized [19–21]. Real-time deep learning has emerged as a powerful tool for enhancing diagnostic accuracy and efficiency. Deep learning models can analyze complex medical data such as imaging scans, genetic information, and patient records to aid in the identification of diseases and conditions. Real-time processing capabilities enable the analysis of incoming data streams, facilitating rapid decision-making and timely interventions [22–24].

Biomarkers have garnered significant attention as potential tools for differentiating between stroke and non-stroke conditions. For instance, elevated levels of specific biomarkers such as brain natriuretic peptide (BNP) and D-dimer have been associated with an increased risk of stroke, serving as valuable indicators in the diagnostic process. Furthermore, ongoing research is exploring the utility of novel biomarkers in discriminating between different stroke subtypes and non-stroke etiologies [25].

Several clinical scales and scores have been developed to aid in the rapid assessment and differentiation of stroke from non-stroke conditions. The National Institutes of Health Stroke Scale (NIHSS) is widely used to quantify stroke severity and guide treatment decisions, while the Cincinnati Prehospital Stroke Scale (CPSS) enables prehospital providers to identify potential stroke cases with high specificity [26]. These tools contribute to streamlining the diagnostic process and facilitating intervention. The emergence of telemedicine has revolutionized the delivery of acute stroke care, allowing for remote assessment and diagnosis. TeleStroke networks leverage audiovisual communication to connect stroke specialists with healthcare facilities, enabling timely evaluation of patients presenting with stroke symptoms [27]. This approach has proven instrumental in extending expert guidance to underserved areas, ultimately improving access to accurate diagnosis and treatment.

Advancements in machine learning and deep learning have paved the way for innovative diagnostic tools in the realm of stroke care. AI algorithms trained on large datasets can analyze medical images and clinical data to differentiate between stroke and non-stroke conditions with high accuracy [28]. Moreover, AI-based decision support systems hold potential for enhancing the efficiency and precision of stroke diagnosis, heralding a new era of personalized medicine. Exploration of genomic and proteomic profiles has unveiled valuable insights into the pathophysiology of stroke and non-stroke conditions. Genetic variations have been linked to an increased susceptibility to certain types of strokes, underscoring the potential utility of genetic testing in risk stratification and differential diagnosis. Furthermore, proteomic profiling offers a window into the molecular signatures associated with different disease states, presenting opportunities for developing novel diagnostic biomarkers [29].

3 Proposed algorithm design methodology

In the experimental setup for real-time stroke detection, a combination of advanced technologies of deep learning is utilized to improve accuracy and maintain privacy. We examined several architectures of YOLOv8 models, which is a cutting-edge neural network architecture targeted for real-time object detection. The proposed models are utilized to identify stroke based on the facial paralysis and non-stroke condition from the faces of the individuals in the images. In this research, federated learning, a decentralized training method, is employed to train the model collaboratively across multiple clients while ensuring the confidentiality of sensitive proposed data. The setup involves a central server overseeing the training process and coordinating interactions

with three different clients, each providing valuable data to enhance the model's performance. To optimize the system's capabilities, NVIDIA platforms are utilized to assess the deployed model's inference. We expedite the assessment of proposed models, facilitating real-time stroke detection and diagnosis. This ensures that the system can efficiently process the recognition of the faces of individuals, promptly identifying signs of stroke and enabling timely medical intervention. Figure 1 illustrates the proposed framework for utilizing real-time deep learning models for stroke and non-stroke detection.

3.1 Dataset collection and labeling

The data used in this research consists of two different categories of images. One group includes individuals diagnosed with acute stroke, while the other group of individuals without such a diagnosis. In total, the dataset comprises 3745 images, offering a substantial pool for analysis. Table 1 provides the main description for the proposed dataset of stroke/non-stroke.

To enhance the efficacy of the model, various data augmentation methods were implemented. These techniques serve to expand the dataset by generating modified versions of the existing images. Specifically, augmentation involved operations such as flipping, rotating, and scaling the images. By applying these transformations, the dataset becomes more diverse and resilient, mimicking a broader range of real-world scenarios. By augmenting the dataset in this manner, the model's accuracy is expected to improve. The varied representations provided by augmented data can help the model generalize better to unseen instances, thereby enhancing its performance when applied to classify images of individuals with or without

Table 1 The main description and parameters for stroke and non-stroke dataset

Parameters	Specifications
Image type	JPG
Images with stroke	1245
Images with non-stroke	2500
Total images	3745

acute stroke. In essence, data augmentation enriches the dataset, enabling the model to learn from a wider spectrum of image variations and complexities. Figure 2 illustrates the distribution for the two classes for the proposed dataset. These visual aids are invaluable for evaluating model performance, detecting class imbalances or dependencies, and guiding decisions related to model refinement and data handling strategies. Class frequency pertains to the distribution of data points among different categories or classes within a dataset. When considering normalized height and width space for stroke and non-stroke datasets, it indicates the number of data points falling within specific height and width ranges. Typically, labels denote the assigned categories or classes for each data point, distinguishing between stroke and non-stroke instances in relevant datasets. A height-width graph typically illustrates data point distributions based on their height and width values, often presented as a scatter plot with height on the x -axis and width on the y -axis. Alternatively, an x - y graph can visualize the data, particularly if additional features beyond height and width are involved, with each data point representing a point in a multi-dimensional space where each axis corresponds to a different feature.

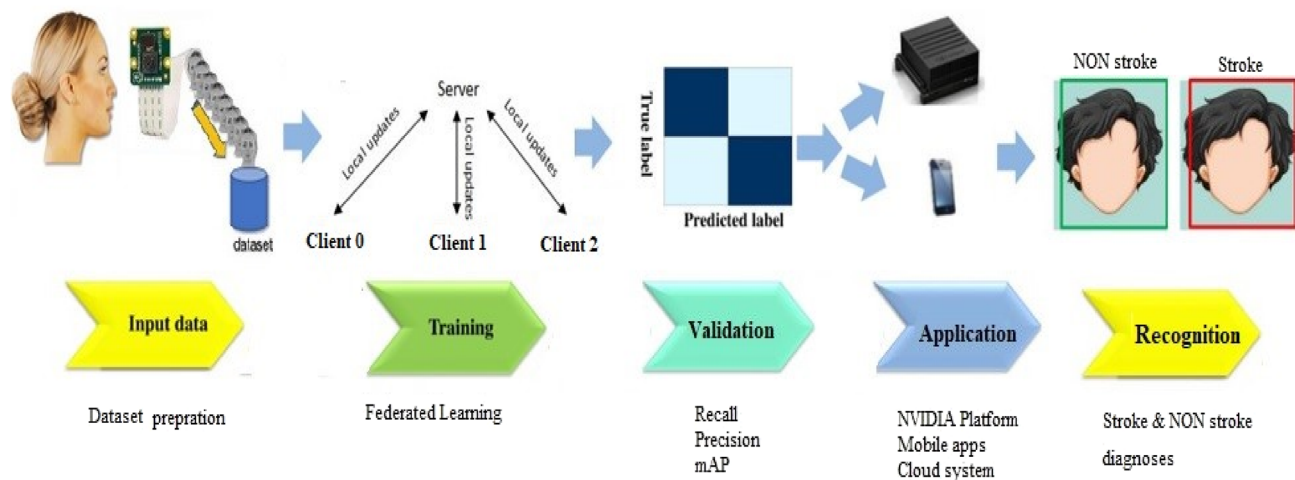
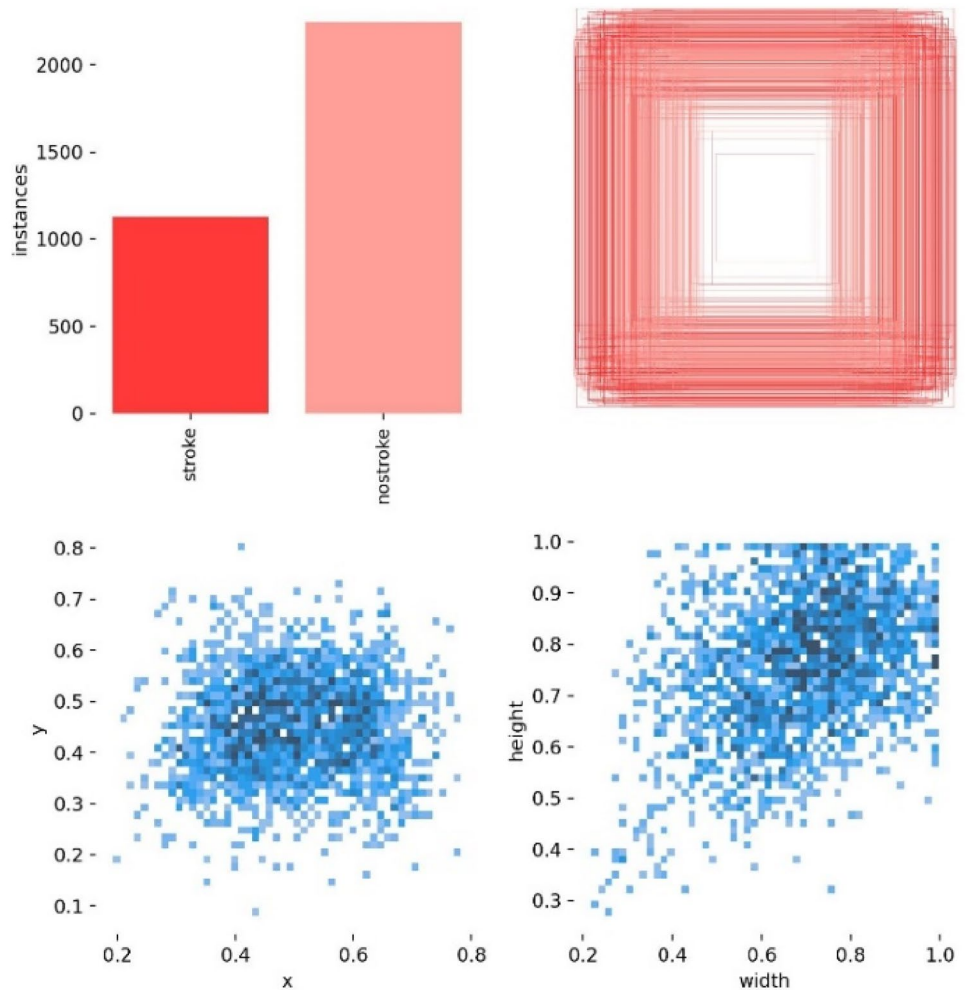


Fig. 1 The proposed system for utilizing real-time deep learning model and federated learning for stroke and non-stroke detection

Fig. 2 Visualization of class distribution in the proposed dataset, with the upper bar graph showing a higher number of instances for 'nostroke' than 'stroke', and the lower scatter plots displaying the density and spread of feature coordinates (x , y) and bounding box dimensions (width, height) for each class



3.2 The proposed object detection architecture

The YOLOv8 architectures mark the forefront of the YOLO series, specializing in the real-time detection of individual faces. YOLOv8 architecture excels in real-time object detection due to its faster speed, increased accuracy, and anchor-free design. This model leverages computer vision, neural networks, deep learning, and image processing techniques to deliver outstanding object recognition capabilities. YOLOv8's multi-scale prediction and enhanced backbone network further boost its object detection performance. Its ability to handle diverse datasets and ease of training make it a versatile tool for various applications. With faster and more precise results, YOLOv8 stands out as a powerful solution in the fields of artificial intelligence and computer vision. It represents the pinnacle of progress in the YOLO object detector lineage, offering unparalleled precision and speed. YOLOv8 models build upon the legacy of their forerunners by integrating groundbreaking features and enhancements, making it the go-to choice for a broad spectrum of object detection challenges across

different settings. It leverages advanced backbone and neck architectures to improve feature extraction significantly, thus boosting the overall efficiency of object detection. With the introduction of an anchor-free split Ultralytics head, it achieves remarkable accuracy improvements and streamlines the detection workflow, outperforming conventional anchor-based methods. YOLOv8 strikes a perfect harmony between speed and accuracy, making it ideal for real-time detection needs in various application areas. It provides a wide range of pre-trained models, facilitating the selection of the most suitable model for specific requirements.

As the most compact variant in the YOLOv8 series, YOLOv8 is engineered for a wide array of detection tasks, from simple object identification to more complex challenges such as instance segmentation, detection of key points, orientation of objects, and categorization. The architecture is a refined version of the CSPDarknet53, incorporating 53 convolutional layers and cross-stage partial connections to bolster the flow of information between layers. The YOLOv8 head consists of several convolutional layers leading to fully connected layers, tasked

with predicting bounding boxes, objectness scores, and class probabilities for identified objects. A distinct feature of YOLOv8 is its self-attention mechanism within the head, enabling the model to focus on specific parts of an image and adjust feature significance according to the relevance of the task. Remarkably adept at detecting objects across multiple scales, it uses a feature pyramid network to identify objects of diverse sizes and scales within an image effectively. This multi-layered pyramid facilitates the detection of both large and small objects, underscoring YOLOv8's adaptability and thoroughness in object detection. The architecture and capabilities of YOLOv8 are visually depicted in Fig. 3.

In the architecture's initial setup, the 6×6 convolution in the stem is replaced with a 3×3 convolution, altering the primary building block, and substituting C2f for C3 as shown in Fig. 4. A summary of the module is provided in the accompanying image, indicating "f" as the feature count, "e" as the expansion rate, and CBS representing a structure that includes a Convolution, Batch Normalization, and a SiLU operation. In the C2f configuration, outputs from all Bottleneck stages (which consist of two 3×3 convolutions connected by residuals and are elaborately termed) are merged. Conversely, C3 utilizes only the output from the final Bottleneck stage. This Bottleneck component mirrors the one found in YOLOv5, with the exception that the initial convolution's kernel size has been

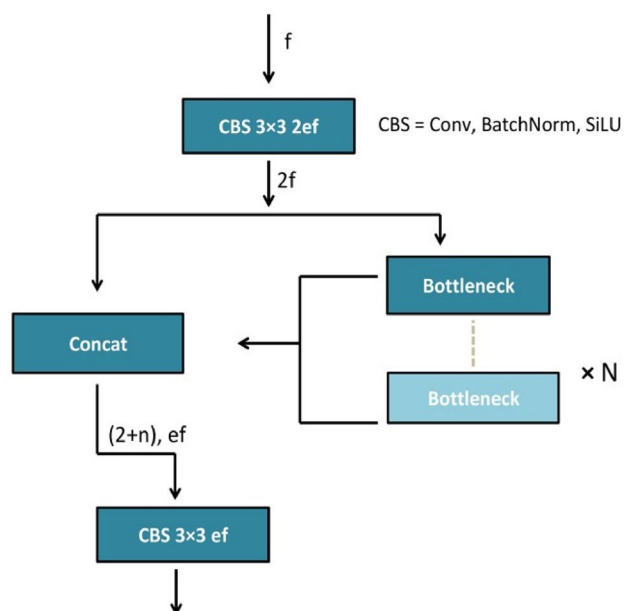
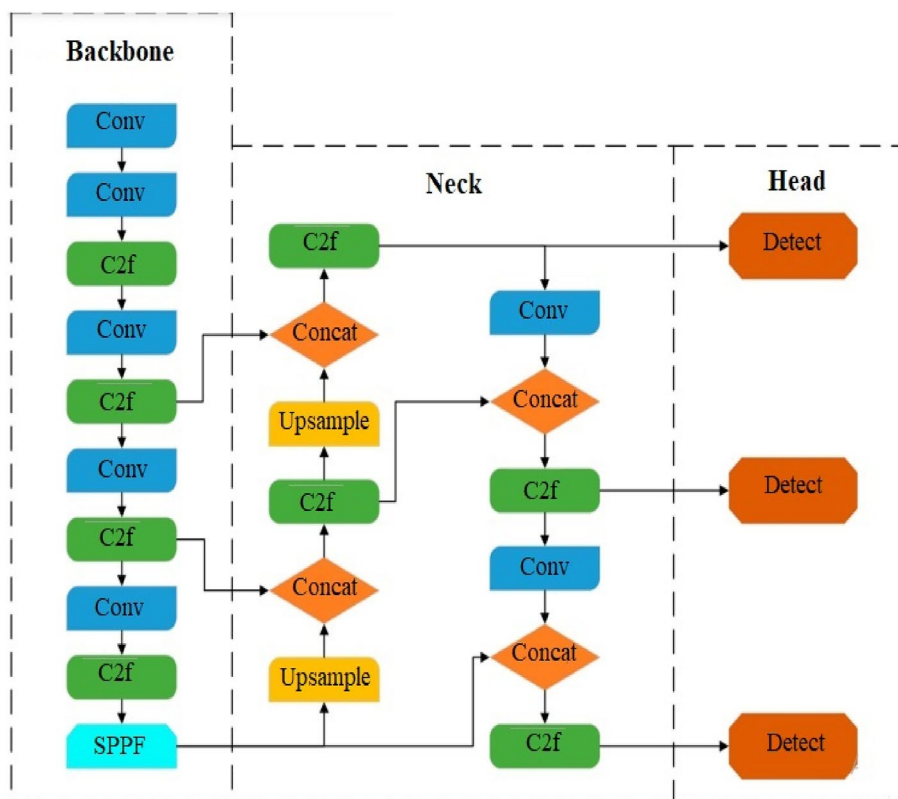


Fig. 4 The C2f layer for YOLOv8 architecture

altered from 1×1 to 3×3 . This change demonstrates that YOLOv8 is gradually returning to the use of the ResNet block.

Within the network's neck section, feature concatenation occurs directly, bypassing the need for uniform channel

Fig. 3 The YOLOv8 architecture incorporates a modified CSPDarknet53 as its core framework. It introduces the C2f module as a replacement for the CSPLayer found in YOLOv5, enhancing its structure. To speed up the computation process, it employs a spatial pyramid pooling fast (SPPF) layer, which consolidates features into a uniform-size map [15]



dimensions. This approach serves to diminish both the parameter count and the overall tensor sizes, streamlining the network's complexity.

Table 2 presents a comparison among different versions of YOLOv8: YOLOv8x, YOLOv8l, YOLOv8m, YOLOv8s, and YOLOv8n, which vary in complexity and performance.

All YOLOv8 versions are designed to process images with an input size of 640 pixels, maintaining consistency in image resolution across models. The complexity and capacity to learn from data are indicated by the number of parameters (Params), measured in millions (M), and the computational workload during inference is represented by floating-point operations per second (FLOPs), measured in billions (B). YOLOv8x, the most complex model, has 68.2 million parameters and requires 257.8 billion FLOPs, which has the highest capacity for learning and computational complexity. Conversely, YOLOv8n is the simplest with only 3.2 million parameters and 8.7 billion FLOPs, indicating it is the least complex and has the lowest computational demand. This gradient of complexity and performance from YOLOv8n to YOLOv8x allows for a range of applications, from lightweight deployments to scenarios requiring high accuracy and computational resources.

3.3 Federated learning

In this research, federated learning (FL) was employed to train the proposed model. FL is a decentralized machine learning method enabling multiple clients to jointly train a model without sharing raw data. Our objective was to emulate a FL setup with 3 clients operating on a single machine, where both the server and all 3 clients reside, sharing CPU, GPU, and memory resources. With 3 clients, we maintain 3 instances of FlowerClient in memory. Executing this on a single machine can strain available memory resources, even if only a subset of clients engages in a FL round. Leveraging the FLOWER platform, clients and the server with overlapping data contribute to model training. Notably, FLOWER boasts an efficient communication protocol, transmitting only model updates, not raw data, thereby significantly reducing communication overhead. This feature renders it

ideal for scenarios with limited bandwidth or high-latency connections. Moreover, FLOWER ensures that raw data remains on local devices, with only model updates, in the form of weight differentials, shared with the central server, safeguarding sensitive information against data breaches and unauthorized access.

Our proposed approach implements an FL system for object detection utilizing the YOLOv8n model, comprising 3 clients and a server. Each client is equipped to train a YOLOv8n on local data, employing the Ultralytics library, with the (*DetectionTrainer*) class managing training specifics. Each client possesses its dataset located in the "data/clients/{idx}" directory, periodically training its local model and computing metrics such as mAP, recall, precision, and loss. Subsequently, during the FL process, the client transmits its model parameters to the server. The server orchestrates the FL process within the Flower framework, aggregating model updates from the 3 clients utilizing the Federated Averaging (*FedAvg*) strategy provided by the *flwr* library. The FL process iterates for a specified number of epochs, with the server engaging in communication with the 3 clients. Following each round, the server aggregates model updates, evaluates the federated model, and logs the results in the "results" directory. Its primary responsibility entails overseeing and coordinating the training process, supervising the participation of the 3 clients in the FL process. These clients, structurally similar, differ mainly in the data they possess, each holding data pertaining to distinct classes of stroke and non-stroke cases. The following four phases outline the steps in a distributed deep learning workflow: from receiving and updating models to local training, evaluation, and secure transmission of improved data back to the server, ensuring a refined and robust predictive performance:

1. **Receive updated model:** Clients establish a secure connection with the server, ensuring data integrity and privacy, through which the server transmits the latest version of the global machine learning model, utilizing efficient communication protocols to minimize latency.
2. **Local model training:** Clients preprocess their local datasets, addressing outliers, missing values, and performing feature scaling as required to enhance data quality. Leveraging their computational resources, clients undertake model training locally, employing fine-tuning such as mini-batch gradient descent or federated learning to effectively handle the datasets
3. **Model evaluation:** After completing the model training phase, clients proceed to conduct thorough evaluations on the trained model, employing diverse metrics including accuracy, precision, recall, and *F1* score, customized to suit their particular application requirements. To ensure robustness and reliability,

Table 2 Comparison of various YOLOv8 models, categorized by their model size, the number of parameters (in millions M), and floating-point operations per second (FLOPs) in billions B

Model	Size (pixels)	Params (M)	FLOPs (B)
YOLOv8x	640	68.2	257.8
YOLOv8l	640	43.7	165.2
YOLOv8m	640	25.9	78.9
YOLOv8s	640	11.2	28.6
YOLOv8n	640	3.2	8.7

clients meticulously analyze the model's predictions on their local datasets, conducting detailed error analysis to pinpoint instances of incorrect predictions and iteratively enhance the training process, thereby refining the model's predictive capabilities and overall performance.

4. **Send updated weights:** Upon completion of local model training and evaluation, clients securely transmit the updated model weights back to the server for aggregation, ensuring data confidentiality. The server aggregates the received model updates from multiple clients using techniques such as federated averaging while synthesizing a new global model. Subsequently, the server conducts additional validation checks on the aggregated model to verify its integrity and stability before deploying it for further inference or subsequent rounds of training.

3.4 Training

The training procedure for YOLOv8 models entails optimizing several hyperparameters to achieve effective object detection. Table 3 illustrates the hyperparameters for tuning YOLOv8 models. The image size, set to 640 pixels, determines the dimensions of input images, crucial for feature extraction and localization. Batch size, specified as 32 defines the number of samples processed before updating the model's parameters, affecting computational efficiency and convergence stability. With 30 epochs, the training duration is partitioned into iterations over the entire dataset, refining the model's performance iteratively. Mosaic, at 0.8, incorporates a data augmentation technique blending multiple images to enhance generalization and robustness. Mixup, assigned a value of 0.2, further diversifies the dataset by linearly interpolating between pairs of images and their labels, augmenting training data diversity. The learning rate, set at 10^{-5} , regulates the step size in updating model parameters, influencing convergence speed and optimization quality. Utilizing the AdamW optimizer, the training algorithm adjusts model weights to minimize the defined loss function effectively. Lastly, the cache parameter, set as False, controls whether to cache datasets in memory, affecting

training speed and memory consumption. Overall, fine-tuning these hyperparameters orchestrates the training process to yield a YOLOv8 model optimized for object detection tasks. All the experiments have been conducted using AWS EC2 G4 instances equipped with 8 NVIDIA T4 GPUs, featuring 96 CPUs and a network bandwidth of 100 Gbps. Table 3 shows the hyperparameters for YOLOv8 training and tuning.

4 Experiment results and discussion

4.1 Evaluation matrices

To evaluate the proposed models, we meticulously performed assessments focusing on key metrics: recall, precision, $F1$ score, and mean Average Precision (mAP). Recall, the measure of a model's ability to identify all relevant instances, is crucial for understanding its sensitivity. Precision, on the other hand, evaluates how many of the identified instances are relevant, highlighting the model's accuracy. The $F1$ score harmonizes recall and precision, providing a single metric to assess a model's balance between sensitivity and accuracy. Lastly, mAP offers a comprehensive evaluation of the model's performance across different thresholds, encapsulating its ability to rank instances correctly. For each metric, equations play a pivotal role in quantification. Recall is calculated as the ratio of true positives to the sum of true positives and false negatives, precision as the ratio of true positives to the sum of true positives and false positives, $F1$ score as the harmonic mean of precision and recall, and mAP as the average of the precision scores at different thresholds, offering a nuanced view of the model's performance. mAP evaluates the balance between precision and recall, providing a comprehensive view of an algorithm's effectiveness across various thresholds. Refer to Eqs. (1–5) for details, where $\lambda(k)$ represents the number of queries and $\lambda(AP_{\lambda})$ denotes the average precision for a specific query λ . These metrics collectively offer a robust framework for evaluating the proposed models, ensuring a comprehensive analysis of their effectiveness:

$$Precision = \frac{TP}{TP + FP} \quad (1)$$

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

$$F1 = 2 \times \frac{(P \times R)}{(P + R)} \quad (3)$$

$$Average\ Precision\ (AP) = \int_{r=0}^1 p(r) dr \quad (4)$$

Table 3 Hyper-parameters for tuning YOLOv8 models

Hyperparameter	Value
Image size	640
Batch size	32
Number of epochs	30
Mosaic	0.8
Mixup	0.2
Learning rate	10^{-5}
Optimizer	AdamW
Cache	False

$$mAP = \frac{1}{k} \sum_i^k AP \quad (5)$$

4.2 Results of the proposed YOLOv8 models

Figure 5 illustrates the performance for the five YOLOv8 models across four metrics which include precision, recall, $F1$ score, and mean average precision (mAP). All models demonstrate a rapid improvement in the first few epochs, indicating fast learning in the early stages. For YOLOv8l and YOLOv8m, precision, recall, and $F1$ scores plateau near the maximum value of 1.0, which illustrates that these models are very accurate and maintain a strong balance between precision and recall. Their mAP, a measure of precision across different recall levels, also levels off high, indicating these models are consistently reliable across both classes. The YOLOv8n figures show a more dramatic learning curve with significant improvements in all metrics until stabilizing. However, the values for all metrics are lower than in the other models, especially the mAP, which illustrates this model has a lower overall object detection performance. For YOLOv8s, the figure indicates a similar trend to YOLOv8l and YOLOv8m, with all metrics reaching and maintaining high values, implying it is quite effective and stable. Lastly, YOLOv8x displays a high performance with all metrics, closely resembling the trends seen in YOLOv8l and YOLOv8m, signifying a robust model with high precision and recall.

Overall, except for YOLOv8n which appears slightly less effective, the models quickly achieve and maintain high precision, recall, and $F1$ scores, with mAP also indicating strong predictive power across different thresholds. YOLOv8n is a simplified model prioritizing speed and efficiency, with fewer parameters and computational needs, leading to lower accuracy compared to more complex models such as YOLOv8x, YOLOv8l, YOLOv8m, and YOLOv8s. These larger models, with more parameters and higher computational demands, are more capable of complex pattern recognition, resulting in better performance on key metrics such as precision and recall. While YOLOv8n is beneficial for fast processing in resource-constrained environments, the larger models are preferable for tasks requiring high accuracy where resources are abundant.

In this research, we have compared the proposed architectures (YOLOv8 models) with YOLOv5n, YOLOv7, and YOLOv7x, all trained for the same number of epochs (30) to ensure a fair comparison. The experimental results indicated that YOLOv5n performed more effectively than both YOLOv7 and YOLOv7x. However, the YOLOv8 models demonstrated superior performance over the previously mentioned architectures, excelling in multiple key metrics

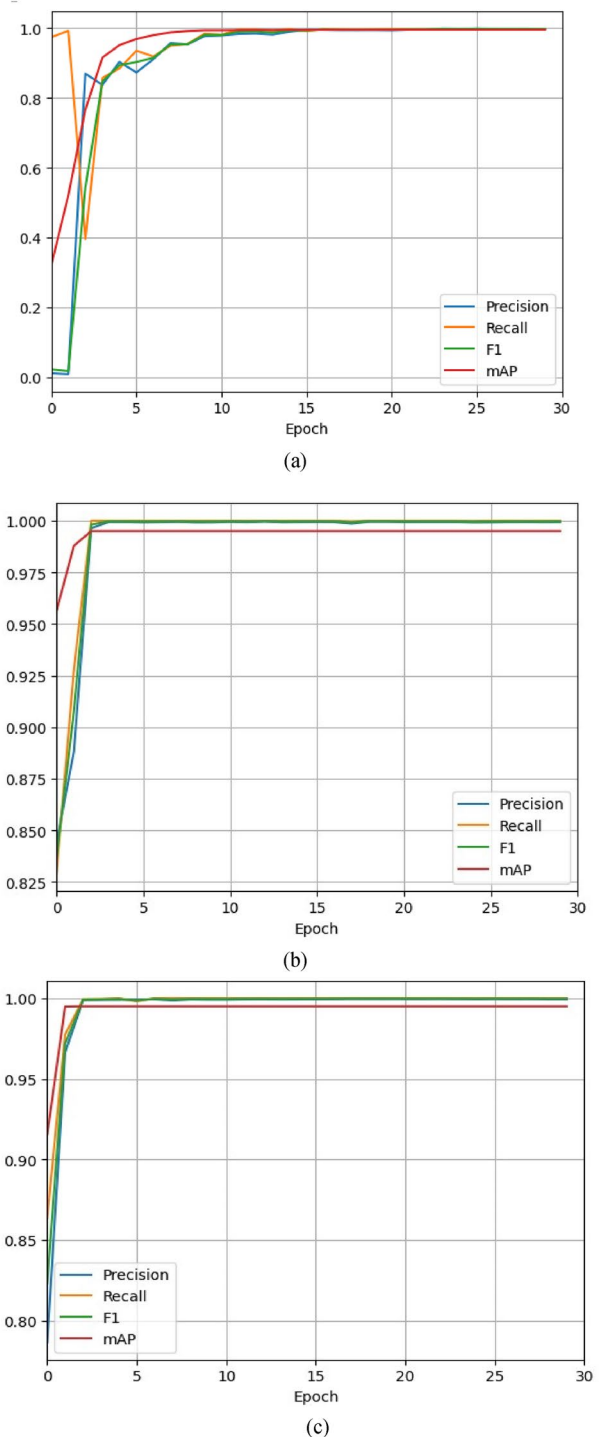
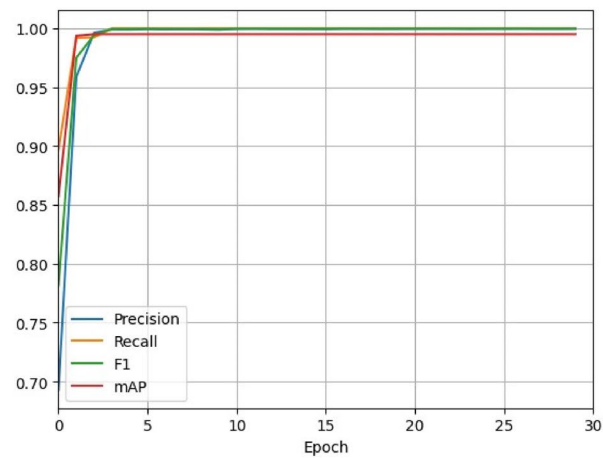
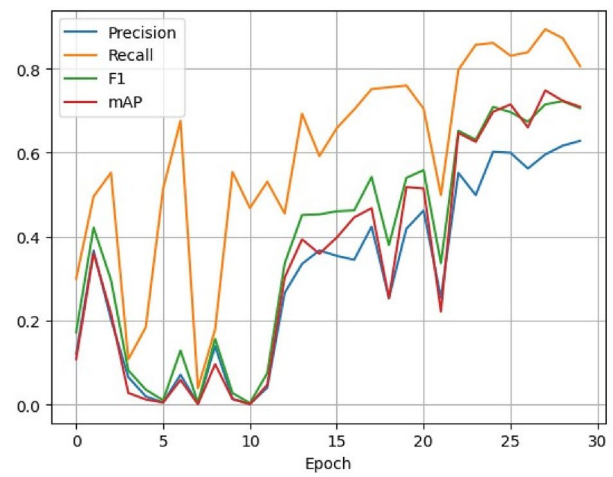


Fig. 5 The performance of YOLOv8 models versus other YOLO architectures, which illustrates recall, precision, mAP, and $F1$: **a** YOLOv8n, **b** YOLOv8l, **c** YOLOv8m, **d** YOLOv8s, **e** YOLOv8x, **f** YOLOv5n, **g** YOLOv7, **h** YOLOv7x

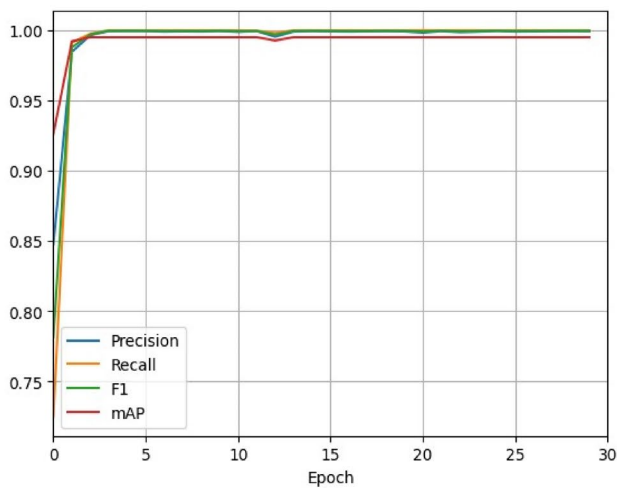
including mean Average Precision (mAP), Recall, Precision, and $F1$ score, see Fig. 5. This enhanced performance underscores the advancements in the YOLOv8 models, making them a suitable choice for stroke and non-stroke application.



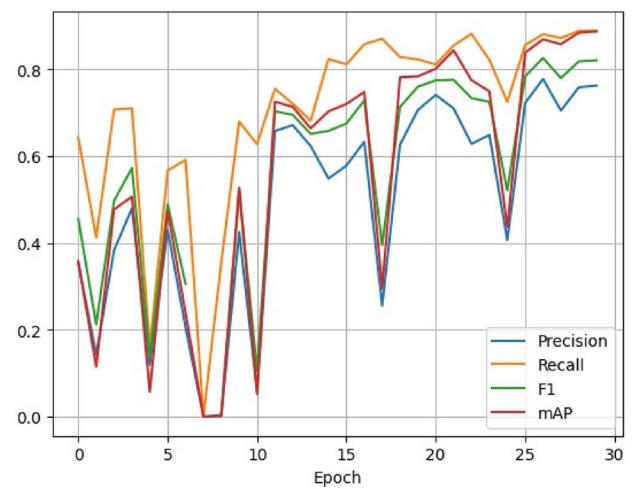
(d)



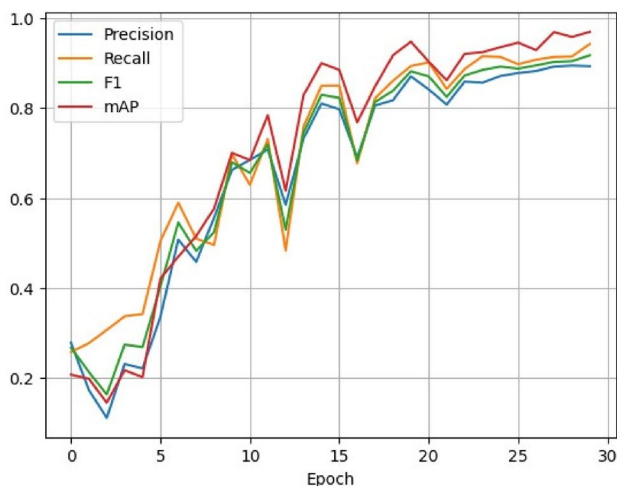
(g)



(e)



(h)



(f)

Fig. 5 (continued)

Fig. 5 (continued)

4.3 Results of federated learning

Federated learning is a machine learning approach where a model is trained across multiple decentralized edge devices (clients) without sharing data, improving privacy, and reducing data centralization. The server coordinates the process, aggregating the client models to form a global model. Here is a detailed analysis of each performance metric across the server and clients: Mean Average Precision (mAP): the first figure shows the mAP metric, which is a measure of precision across recall levels and is commonly used in ranking tasks, object detection, and information retrieval. It appears that the server and all clients quickly improve performance within the first five rounds, indicating a rapid learning rate in the early stages of training. After around five rounds, the server and client 0 show stable performance, while client 1 and client 2 have some variability but generally maintain a high mAP.

This could imply that client 1 and client 2 are encountering more complex and variable data than client 0, leading to slight fluctuations in performance. **Precision:** the second figure measures precision, the ratio of true positive predictions to the total number of positive predictions. The precision for all entities climbs sharply within the initial rounds and then plateaus, indicating that the number of false positives did not significantly increase as more instances were classified as positives, which is good. The server maintains the highest precision, which shows that the aggregated model is accurate in its predictions compared to individual clients. **Recall:** the third figure represents recall, which is the ratio of true positive predictions to the total number of actual positives. Recall also increases rapidly at the beginning, which illustrates that the model's has the ability to identify relevant instances with initial training rounds. The server shows the highest recall, indicating it is better at detecting true positives than the clients. Across all metrics, the performance of the server is consistently equal to and better than the clients, which is expected in federated learning because the server model benefits from the aggregated updates from all clients. **Key observations:** rapid learning: all metrics improve significantly in the initial rounds of training, indicating the clients' updates are meaningful and improve the global model quickly. Stabilization: after the initial improvement, all metrics tend to stabilize, with the server typically showing the least variance, which indicates the aggregated model's robustness. Client variability: there is some variability in the clients' performance metrics, particularly in mAP and precision. This could be due to differences in local data distributions, that some clients have data that is not as well represented in the global model. **Convergence:** all clients seem to converge towards the server's performance, especially in recall, which shows that over time, client-specific models are becoming more aligned with the aggregated server model, which is the desired outcome in federated learning for consistency and fairness. Figure 6 shows the performance which includes mean Average Precision (mAP), precision, and recall scores across multiple rounds of training for 3 clients and server in a federated learning system.

5 Real-time edge execution techniques

NVIDIA's Jetson platform has a series of powerful and efficient AI computing devices designed to bring deep learning to the edge. The series encompasses a range of hardware tailored to various performance needs and power constraints, making AI accessible across a broad spectrum of applications, from robotics to embedded systems. At the entry-level, the Jetson Nano stands out as a compact yet capable device, providing a cost-effective solution for projects that require AI but are constrained by power and space. Scaling up, the Jetson Xavier AGX presents a significant leap

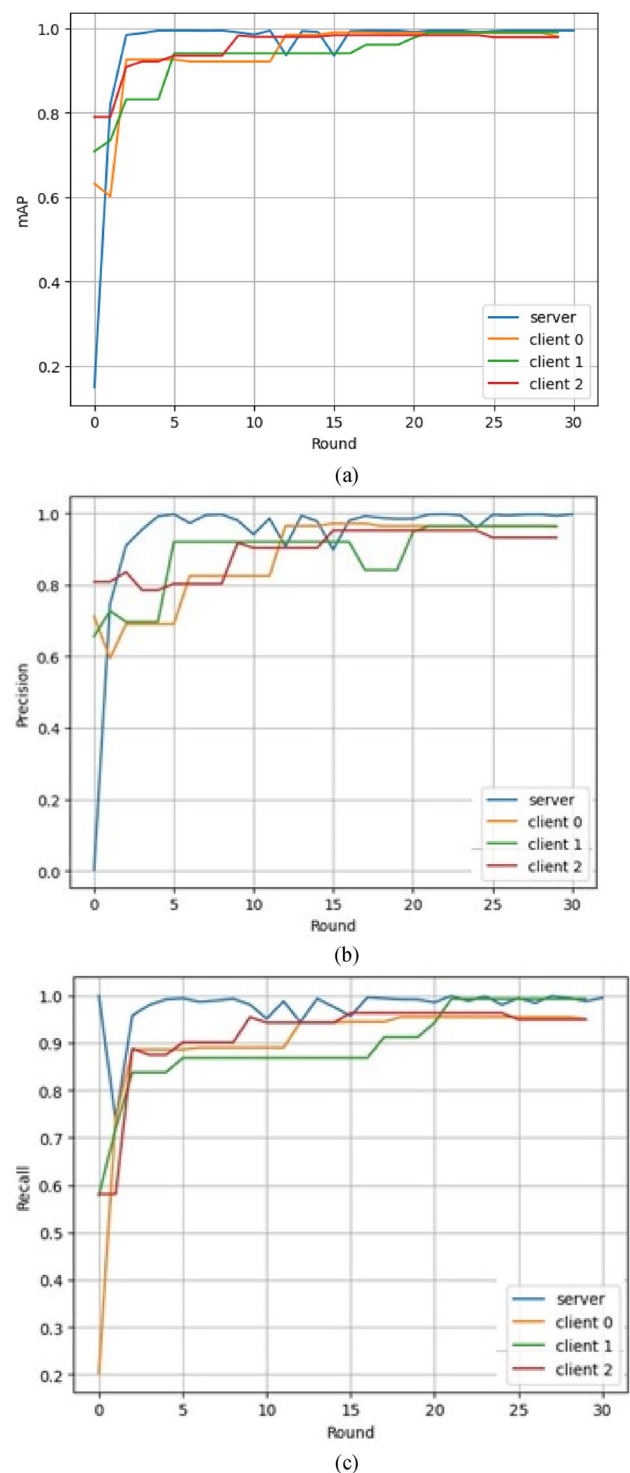


Fig. 6 The performance evolution which include **a** mean Average Precision (mAP), **b** precision, and **c** recall scores across multiple rounds of training for 3 clients and server in a federated learning system

in performance with its advanced GPU architecture and AI accelerators, aimed at more demanding tasks that require higher computational throughput. At the pinnacle is the

Jetson Orin, the latest addition to the series, which represents the cutting-edge in edge AI performance. It offers breakthrough capabilities for autonomous machines, delivering the highest performance and energy efficiency for AI, robotics, and other compute-intensive tasks. Each of these platforms is engineered to provide flexibility, scalability, and ease-of-integration, all crucial for innovators looking to pioneer the next generation of intelligent machines. The graph compares the NVIDIA Jetson Nano, Jetson AGX Xavier, and Jetson Orin platforms, focusing on CUDA cores and AI performance (TOPS). The bar plot shows a significant increase in CUDA cores across the models, indicating enhanced parallel computing capability. The line plot illustrates AI performance, where Jetson Orin demonstrates a substantial leap, reaching up to 200 TOPS, compared to Jetson Nano's 0.5 TOPS and Jetson AGX Xavier's 32 TOPS. This progression underscores the evolution in processing power and AI capabilities, with Jetson Orin offering vastly superior performance for demanding AI applications and computational tasks. Figure 7 presents a comparative analysis of NVIDIA Jetson Platforms, displaying the count of CUDA cores and the AI performance capacity (quantified in TOPS—tera operations per second) among three variants: Jetson Nano, Jetson Xavier AGX, and Jetson Orin AGX.

Figure 8 illustrates a real-time detection frames per second (FPS) comparison among different YOLOv8 models running on three NVIDIA Jetson platforms: Nano, Xavier AGX, and Orin. Across all models, the Jetson Xavier AGX and Jetson Orin outperform the Jetson Nano in terms of FPS, which illustrates that they have more processing power and are more efficient in handling these tasks. We compared the performance of the proposed approach on NVIDIA devices with and without the utilization of CUDA. This comparison underscores the critical importance of using CUDA and GPU acceleration for executing YOLOv8 models to meet real-time requirements. CUDA significantly enhances the speed, enabling the YOLOv8 models to process data at a much faster rate. By leveraging the parallel processing power of GPUs, CUDA reduces the execution time, making it possible to achieve real-time object detection and analysis. From Fig. 8, it is evident that YOLOv8n achieves the highest FPS on all three platforms, indicating it is the fastest model for real-time detection. This aligns with its structure of being a streamlined model that prioritizes speed and efficiency. As a result, despite having lower accuracy metrics, YOLOv8n is highly suitable for applications where real-time processing is crucial. In contrast, YOLOv8x, which is the most computationally intensive model, exhibits the lowest FPS across all platforms. This illustrates that while YOLOv8x has the best accuracy metrics, its complexity makes it less suitable for scenarios requiring high-speed object detection. The other models, YOLOv8l, YOLOv8m, and YOLOv8s, show a gradation of FPS performance, with the more complex models

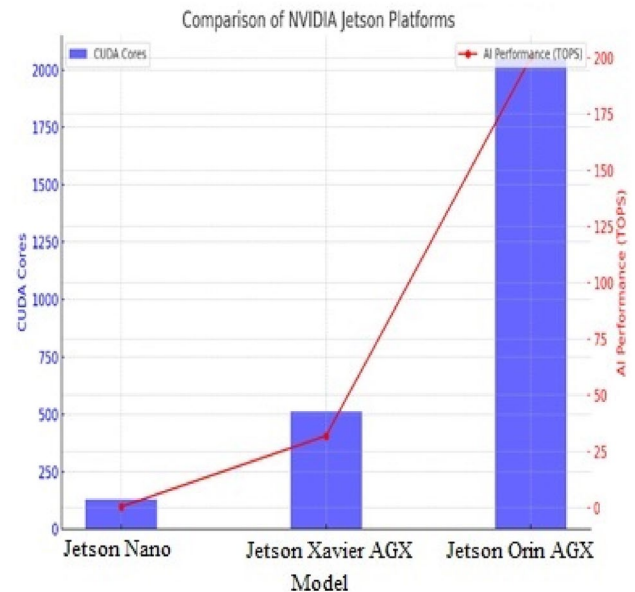


Fig. 7 A comparative analysis of NVIDIA Jetson Platforms, which illustrates number of CUDA cores and AI performance (measured in TOPS—tera operations per second) across three models: Jetson Nano, Jetson Xavier AGX, and Jetson Orin AGX

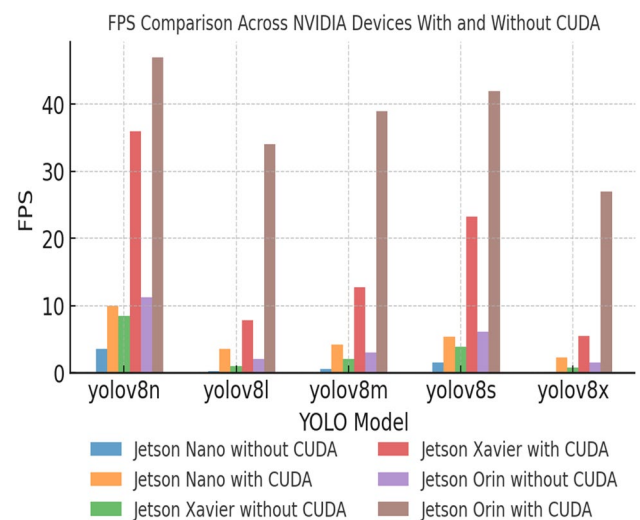


Fig. 8 The frames per second (FPS) performance for different YOLO models (YOLOv8n, YOLOv8l, YOLOv8m, YOLOv8s, YOLOv8x) across three NVIDIA Jetson devices (Nano, Xavier, Orin) with and without CUDA acceleration

(e.g., YOLOv8l) being slower than the simpler ones (e.g., YOLOv8s). The performance on the Jetson Xavier AGX and Jetson Orin is significantly better than on the Jetson Nano, which is the least powerful of the three hardware platforms.

The graph highlights the trade-off between accuracy and speed among the YOLOv8 models and underscores the importance of choosing the right model and hardware

platform based on the specific requirements of the application. Real-time detection of strokes using the proposed deep learning object detection models is crucial for timely medical intervention and improved patient outcomes. The proposed models, in particular YOLOv8n architecture, are designed to quickly and accurately identify stroke indicators from facial features. By processing data in real time, the models can provide immediate feedback to healthcare professionals, enabling rapid decision-making. This rapid analysis is essential in emergency situations where every second counts. The ability to differentiate between stroke and non-stroke cases on the spot helps in prioritizing treatment, reducing the risk of long-term damage, and potentially saving lives.

Power consumption measurement has been carried across the three NVIDIA platforms. The Jetson Nano shows remarkable energy efficiency with consistently low power usage across all YOLO models, making it an ideal candidate for power-sensitive applications. In contrast, the Jetson Orin, while being the most power-hungry, presumably offers superior computational performance, a trade-off that might be justified for demanding tasks where processing speed is crucial. The Jetson Xavier AGX stands in between a moderate balance between power and performance. Notably, the progression from the YOLOv8l to YOLOv8m and further models exhibits a trend of increasing power requirements, which could indicate more computationally intensive processes and greater exploitation of the hardware's capabilities. This increasing trend is most pronounced with the Jetson Orin, which shows a substantial leap in power consumption for the latest models, signifying its capability to unleash the full potential of advanced YOLO models, albeit at a greater energy cost. This data is vital for selecting the appropriate hardware for proposed models where both the performance and the power budget are to be optimized. Figure 9 illustrates the power consumption across various YOLO models when deployed on NVIDIA's Jetson hardware platforms: Nano, Xavier AGX, and Orin.

The temperature data provided for different YOLOv8 models across NVIDIA Jetson platforms Nano, Xavier AGX, and Orin show some interesting trends: Starting with the Jetson Nano, the temperatures for the YOLO models are higher compared to the other two platforms, with YOLOv8x reaching the highest temperature at 54 °C. This demonstrates that while the Nano is power-efficient, as seen in the previous analysis, it does not dissipate heat as effectively. This is due to a less robust cooling system and lower thermal capacity.

However, there is a clear downward trend in temperature from YOLOv8x to YOLOv8n, with the latter running at a cooler 40 °C. This indicates that the newer models are more efficient. The Jetson Xavier AGX shows a consistently cooler operational temperature across all YOLO models when compared to the Jetson Nano, which shows better

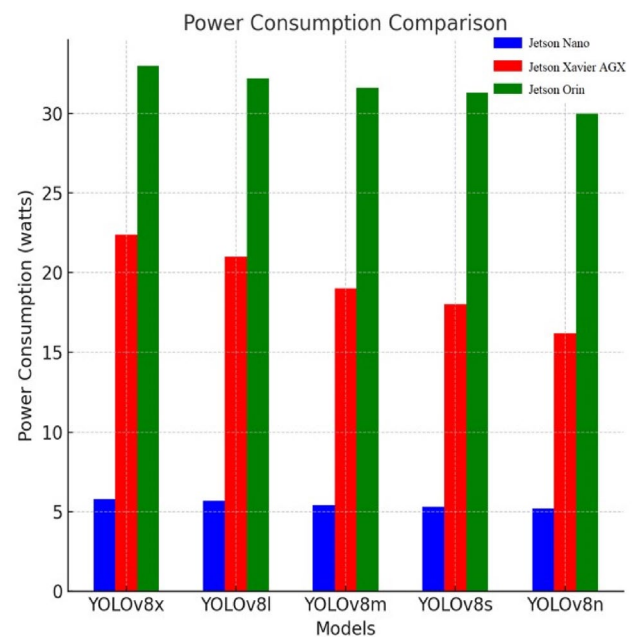


Fig. 9 Comparison of power consumption across NVIDIA Jetson platforms (Nano, Xavier AGX, Orin) running different YOLOv8 models, demonstrating the varying energy requirements of each model on the respective hardware

thermal management. The temperatures range from 43 °C for YOLOv8x to 37 °C for YOLOv8n. The lower temperatures could also imply that the Xavier AGX is more capable of handling the computational demands of the YOLO models efficiently, thereby generating less heat. The Jetson Orin records temperatures similar to the Xavier AGX but is marginally cooler across all models except for YOLOv8s, where it equals the AGX at 39 °C. This could be due to the efficient power usage that translates into less heat output. YOLOv8n runs coolest on the Orin at 36 °C. Tables 4, 5 and 6 provide operation temperature for the executed YOLOv8 model on NVIDIA platforms.

6 Conclusion

This research presented an innovative, real-time stroke detection system by utilization deep learning and federated learning to offer a solution that is both efficient and privacy conscious. By combining the advanced analytical capabilities of deep learning, particularly through the use of YOLOv8 models, with the privacy-preserving features of federated learning, we have developed a system that stands to significantly improve the timeliness and accuracy of stroke detection. This approach mitigates the limitations of traditional stroke detection methods, such as the reliance on manual interpretation which is slow and error-prone, and addresses the challenges of requiring extensive, diverse

Table 4 Operational temperatures of YOLOv8 models on the NVIDIA Jetson nano-platform

Jetson nano	Temperature (°C)
YOLOv8x	54
YOLOv8l	45
YOLOv8m	43.5
YOLOv8s	42.7
YOLOv8n	40

Table 5 Operational temperatures of YOLOv8 models on the NVIDIA Jetson Xavier AGX platform

Jetson Xavier AGX	Temperature (°C)
YOLOv8x	43
YOLOv8l	41
YOLOv8m	40
YOLOv8s	39
YOLOv8n	37

Table 6 Operational temperatures of YOLOv8 models on the NVIDIA Jetson Xavier Orin platform

Jetson Orin	Temperature (°C)
YOLOv8x	42
YOLOv8l	40
YOLOv8m	39
YOLOv8s	39
YOLOv8n	36

datasets and navigating privacy concerns. YOLOv8 models have demonstrated promising results in terms of mean Average Precision (mAP), recall, and precision. However, the YOLOv8n model appears to be slightly less effective. Designed to prioritize speed and efficiency, YOLOv8n is a simplified version with fewer parameters and reduced computational requirements. As a consequence, it achieves lower accuracy compared to the other models. The utilization of NVIDIA platforms for their superior GPU capabilities has enabled real-time processing and analysis, ensuring that the proposed system can function effectively in a clinical setting. The YOLOv8n model has been recognized for its superior real-time detection performance compared to earlier versions. This improvement is mainly because of its lightweight architecture. YOLOv8 models, while excelling in real-time detection on NVIDIA devices, demand high power consumption. This requirement limits their practicality for low-cost embedded devices, highlighting a significant challenge in balancing computational efficiency and performance in resource-constrained environments. Further optimization is needed to make these models viable for

broader applications. The implications of this research are profound, offering a pathway to enhancing patient outcomes by enabling healthcare professionals to make quicker, more informed decisions. Further to our exploration, we will convert the proposed approach to TensorFlow Lite to optimize the architecture. This will enhance our results for real-time detection and reduce the demand for power consumption. In addition to that, we will continue further research and experimentation with the most recent object detection models, such as YOLOv9, which has been released recently.

Acknowledgements This work has been partially supported by the Forelab Dipartimento di Eccellenza project and by the spoke 6 of CN1 on HPC, Big Data and quantum of the PNRR, both by MUR

Author contributions AE carried out the experiments, and wrote the main manuscript text with support from PD and QZ. SS supervised the project.

Funding Open access funding provided by Università di Pisa within the CRUI-CARE Agreement.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Powers, W.J., Rabinstein, A.A., Ackerson, T., et al.: Guidelines for the early management of patients with acute ischemic stroke: 2019 update to the 2018 guidelines for the early management of acute ischemic stroke: a guideline for healthcare professionals from the American Heart Association/American Stroke Association. *Stroke* **50**(12), e344–e418 (2019)
2. Basu, A.P.: Early intervention after perinatal stroke: opportunities and challenges. *Dev. Med. Child Neurol.* **56**(6), 516–521 (2014)
3. Hayes, S.H., Carroll, S.R.: Early intervention care in the acute stroke patient. *Arch. Phys. Med. Rehabil.* **67**(5), 319–321 (1986)
4. Díez-Tejedor, E., Fuentes, B.: Acute care in stroke: the importance of early intervention to achieve better brain protection. *Cerebrovasc. Dis.* **17**(Suppl. 1), 130–137 (2003)
5. Jauch, E.C., Saver, J.L., Adams, H.P., Jr., et al.: Guidelines for the early management of patients with acute ischemic stroke: a

- guideline for healthcare professionals from the American Heart Association/American Stroke Association. *Stroke* **44**(3), 870–947 (2013)
6. Merino, J.G., Warach, S.: Imaging of acute stroke. *Nat. Rev. Neurol.* **6**(10), 560–571 (2010)
 7. Adeoye, O., Hornung, R.: Practical considerations in acute stroke management: update to the 2018 American Heart Association/American Stroke Association guidelines for the early management of acute ischemic stroke [published correction in *Stroke*. 2019 Jan;50(1):e18]. *Stroke* **49**(3), e123–e125 (2018)
 8. Albers, G.W., Marks, M.P., Kemp, S., et al.: Thrombectomy for stroke at 6 to 16 hours with selection by perfusion imaging [published correction appears in *N Engl J Med*. 2018 Mar 22;378(12):1161]. *N. Engl. J. Med.* **378**(8), 708–718 (2018)
 9. Esteva, A., et al.: Dermatologist-level classification of skin cancer with deep neural networks. *Nature* **542**, 115–118 (2017)
 10. McKinney, S.M., et al.: International evaluation of an AI system for breast cancer screening. *Nature* **577**, 89–94 (2020)
 11. Elhanashi, A., Lowe, D., Saponara, S., Moshfeghi, Y.: Deep learning techniques to identify and classify COVID-19 abnormalities on chest X-ray images. In: *Proceedings of SPIE 12102, Real-Time Image Processing and Deep Learning*, p. 1210204 (2022)
 12. Saponara, S., Elhanashi, A., Gagliardi, A.: Reconstruct fingerprint images using deep learning and sparse autoencoder algorithms. In: *Proceedings of SPIE 11736, Real-Time Image Processing and Deep Learning* (2021)
 13. Elhanashi, A., Saponara, S., Dini, P., et al.: An integrated and real-time social distancing, mask detection, and facial temperature video measurement system for pandemic monitoring. *J. Real-Time Image Proc.* **20**, 95 (2023). <https://doi.org/10.1007/s11554-023-01353-0>
 14. Zheng, Q., Zhao, P., Wang, H., Elhanashi, A., Saponara, S.: Fine-grained modulation classification using multi-scale radio transformer with dual-channel representation. *IEEE Commun. Lett.* **26**(6), 1298–1302 (2022). <https://doi.org/10.1109/LCOMM.2022.3145647>
 15. Campbell, B.C.V., De Silva, D.A., Macleod, M.R., et al.: Ischaemic stroke. *Nat. Rev. Dis. Primers.* **5**, 70 (2019). <https://doi.org/10.1038/s41572-019-0118-8>
 16. Elhanashi, A., Saponara, S., Zheng, Q.: Classification and localization of multi-type abnormalities on chest X-rays images. *IEEE Access* **11**, 83264–83277 (2023). <https://doi.org/10.1109/ACCESS.2023.3302180>
 17. Aggarwal, R., Sounderajah, V., Martin, G., et al.: Diagnostic accuracy of deep learning in medical imaging: a systematic review and meta-analysis. *npj Digit. Med.* **4**, 65 (2021). <https://doi.org/10.1038/s41746-021-00438-z>
 18. Varoquaux, G., Cheplygina, V.: Machine learning for medical imaging: methodological failures and recommendations for the future. *npj Digit. Med.* **5**, 48 (2022). <https://doi.org/10.1038/s41746-022-00592-y>
 19. Schünke, L.C., Mello, B., da Costa, C.A., Antunes, R.S., Rigo, S.J., Ramos, G.O., Righi, R.D.R., Scherer, J.N., Donida, B.: A rapid review of machine learning approaches for telemedicine in the scope of COVID-19. *Artif. Intell. Med.* **129**, 102312 (2022). <https://doi.org/10.1016/j.artmed.2022.102312>
 20. El-Sherif, D.M., Abouzid, M., Elzarif, M.T., Ahmed, A.A., Albakri, A., Alshehri, M.M.: Telehealth and artificial intelligence insights into healthcare during the COVID-19 pandemic. *Healthcare (Basel)* **10**(2), 385 (2022). <https://doi.org/10.3390/healthcare10020385>
 21. Christopoulou, S.C.: Machine learning models and technologies for evidence-based telehealth and smart care: a review. *BioMedInformatics* **4**, 754–779 (2024). <https://doi.org/10.3390/biomedinformatics4010042>
 22. Haleem, M.S., Ekuban, A., Antonini, A., Pagliara, S., Pecchia, L., Allocca, C.: Deep-learning-driven techniques for real-time multimodal health and physical data synthesis. *Electronics* **12**, 1989 (2023). <https://doi.org/10.3390/electronics12091989>
 23. Elhanashi, A., Dini, P., Saponara, S., Zheng, Q.: Integration of deep learning into the IoT: a survey of techniques and challenges for real-world applications. *Electronics* **12**, 4925 (2023). <https://doi.org/10.3390/electronics12244925>
 24. Salehin, I., et al.: Real-time medical image classification with ML framework and dedicated CNN–LSTM architecture. *J. Sens.* (2023). <https://doi.org/10.1155/2023/3717035>
 25. Johnson, C.O., et al.: Global, regional, and national burden of stroke, 1990–2016: a systematic analysis for the Global Burden of Disease Study 2016. *Lancet Neurol.* **18**(5), 439–458 (2019)
 26. Katz, B.S., McMullan, J.T., Sucharew, H., Adeoye, O., Broderick, J.P.: Design and validation of a prehospital scale to predict stroke severity: Cincinnati Prehospital Stroke Severity Scale. *Stroke* **46**(6), 1508–1512 (2015). <https://doi.org/10.1161/STROKEAHA.115.008804>
 27. Meyer, B.C., Raman, R., Hemmen, T., Obler, R., Zivin, J.A., Rao, R., Thomas, R.G., Lyden, P.D.: Efficacy of site-independent telemedicine in the STRoke DOC trial: a randomised, blinded, prospective study. *Lancet Neurol.* **7**(9), 787–795 (2008). [https://doi.org/10.1016/S1474-4422\(08\)70171-6](https://doi.org/10.1016/S1474-4422(08)70171-6)
 28. Zhao, J., et al.: Impact of the COVID-19 epidemic on stroke care and potential solutions. *Stroke* **51**(7), 1996–2001 (2020)
 29. Hunt, L.C., et al.: Integrated genomic and proteomic analyses identify stimulus-dependent molecular changes associated with distinct modes of skeletal muscle atrophy. *Cell Rep.* **37**(6), 109971 (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Abdussalam Elhanashi is a researcher at the Università di Pisa, Italy. He received the M.Sc. degree in Electronics and Electrical Engineering from the University of Glasgow in Scotland, and he earned a Master of Business Administration (MBA) from the University of Nicosia in Cyprus. He obtained a Ph.D. degree in Information Engineering from the University of Pisa. Abdussalam was selected as the best candidate across Libya and was awarded a merit-based distinguished scholarship from Islamic Bank Development (IsDB) 2019–2020 to pursue his Ph.D. He was a Research Fellow at the University of Strathclyde in 2021, applying deep learning models to CT-scan and X-ray images. He was a visiting researcher at the Graduate School of Advanced Science and Engineering at Hiroshima University in Japan in 2022. He has authored and co-authored several scientific articles in international conferences and journals. He published the first book in Libya on artificial intelligence in Arabic. He is a developer at the Society for Imaging Informatics in Medicine (SIIM) in the USA. He is a member of IET and IEEE. His current research interests are deep learning, video/imaging processing and coding, surveillance and real-time applications, medical images, low-cost embedded systems, and IoT devices.

Pierpaolo Dini is an Assistant Professor in the Department of Information Engineering at the University of Pisa. He earned the master's degree in Electrical and Automation Engineering (2016) and the Ph.D. degree in Smart Industry (2021), both with honors, from the University of Pisa. His research focuses on the European project Hi-Efficient, collaborating with Vrije Universiteit Brussel (VUB) and industry partners FLAG-MS and Ideas & Motion on the design, verification, and embedded integration of control and monitoring algorithms for complex power electronic systems. He also engages in R&D activities with

international companies, such as Marelli, Magna, Vitesco, Continental, and Toyota, focusing on control and monitoring system development and integration for industrial applications. He is co-inventor of two patents related to the safety of CAN-bus in-vehicle networking. He has authored over 30 scientific articles, with an H-index of 15 and over 400 citations.

Sergio Saponara is the IEEE Distinguished Lecturer, obtained his master's degree cum laude and Ph.D. degree in Electronic Engineering from the University of Pisa. In 2002, he was a Marie Curie Research Fellow at the Inter-University Microelectronics Center (IMEC) in Leuven, Belgium. He was also a post-graduate researcher at the National Research Council. Currently, he is a Full Professor of Electronics at the University of Pisa, where he teaches courses in Vehicular Electronics, Electronic Systems for Robotics, HW and Embedded Security for the Master's degrees in Vehicle Engineering, Robotics and Automation Engineering, and Cybersecurity. Additionally, he teaches Electronics at the Italian Naval Academy in Livorno. Prof. Saponara is the President of the Bachelor's and Master's programs in electronic engineering at the University of Pisa. He has co-authored numerous scientific articles and patents. He is a Founding Member of the IoT CASS SiG and has been a Technical Program Committee (TPC) Member for over 100 international IEEE and SPIE conferences. He also serves as the Director of the summer school "Enabling Technologies for the Internet of Things (IoT)" and the specialization course "Automotive Electronics

and Powertrain Electrification". Additionally, he is the Vice Director of the Inter-university Center for Automotive Research (UCAR).

Qinghe Zheng is an Associate Professor of the Shandong Management University. He received the B.E. degree from Xi'an University of Posts and Telecommunications in 2014. He received the M.E. and Ph.D. degrees from Shandong University in 2018 and 2022, respectively. He has coauthored about 50 peer reviewed scientific journal articles and holds 11 patents. He is a member of IEEE and Guest Editors of several journals, including *Sensors*, *Symmetry*, and *Electronics*. He is also TPC members of some international academic conferences, including ICAML 2020, ICBTA 2020, BDMIP 2020, IVP AI 2020, ICCAES 2021, etc. His research interests include image processing, pattern recognition, deep learning, and edge computing.